

1. 软件开发四大本质困难和挑战

- 复杂性、不可见性、可变性、一致性

2. 管理三大关键要素

- 目标、状态、纠偏

3. 典型的三大目标

- 成本、质量、工期

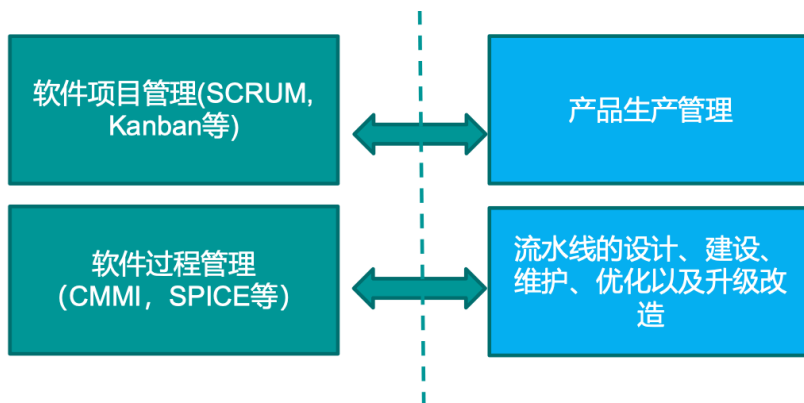
4. 生命周期模型和软件过程（背）

- 软件过程：是为了实现一个或者多个事先定义的目标而建立起来的一组实践的集合，这组实践之间往往有一定的先后顺序，作为一个整体来实现事先定义的一个或者多个目标
- 生命周期模型：对软件过程的一种人为的划分，比如：瀑布模型、迭代式模型、增量模型、螺旋模型、原型法等等
- 区别：
 - 生命周期模型是对一个软件开发过程的**人为划分**
 - 生命周期模型是软件开发过程的**主框架**，是对软件开发过程的一种**粗粒度划分**
 - 生命周期模型往往**不包括技术实践**

5. 软件项目管理和软件过程管理是两回事（背）

- 管理对象是软件过程

- 管理的**目的**是为了让软件过程在开发效率、质量等方面有着更好性能绩效（performance）



管理视角的核心问题——能否复制成功？

- 软件过程管理和软件过程改进意思相近
 - 软件过程管理参考模型 CMM/CMMI, SPICE等
 - 软件过程改进参考元模型 PDCA, IDEAL

- 软件项目管理是应用**方法、工具、技术**以及**人员能力**来完成软件项目、实现项目目标的过程
- 软件过程管理是为了让软件过程在**开发效率、质量**等方面有着更好性能绩效

6. 软件发展三大阶段以及典型的软件开发方法和实践（背）

- 软硬件一体化阶段（50 ~ 70 年代）
 - 线性顺序过程，事实上是硬件开发流程

- Measure twice, cut once
- Code and fix
- 软件成为独立的产品（70 ~ 90 年代）
 - 结构化程序设计和瀑布模型
 - 成熟度模型
- 网络化和服务化（90 年代中期至今）
 - 迭代式开发
 - 敏捷开发（XP、SCRUM、Kanban）
 - 开源软件开发方法
 - DevOps

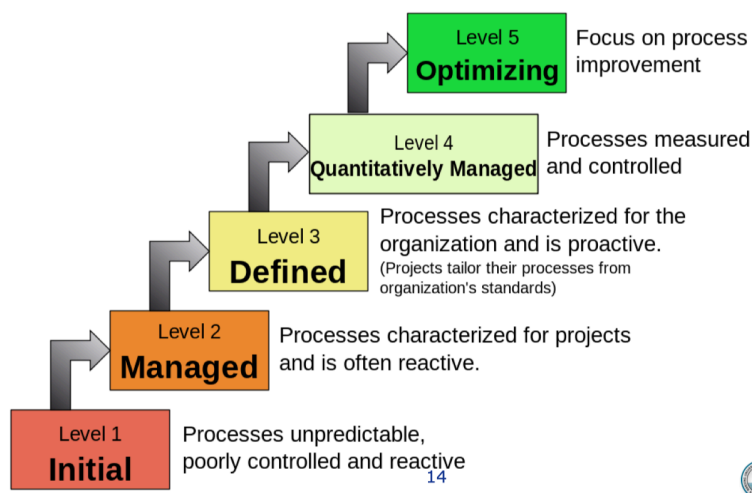
如何理解瀑布模型？（背）

1. 瀑布模型不是单一模型，是一系列模型，覆盖最简单场景（过程元素少）到最复杂的场景（过程元素多）
2. 软件项目应该结合实际情况选择合适过程元素的瀑布模型，基本原则是，项目面临困难和挑战越多，选择的模型应该越复杂
3. 软件项目团队往往低估项目的挑战，选择了过于简单的不使用瀑布模型

7. CMMI 成熟度模型（背）

CMMI = Capability Maturity Model Integration

Characteristics of the Maturity levels



CMMI-DEV V1.3 版本五个不同的成熟度等级分别是什么？为什么四级和五级被称为高等级？与普通等级的本质差别是什么？

- 等级 1：初始级 Initial
 - 开发相对混乱，依赖个人英雄主义，没有过程概念，救火文化盛行
- 等级 2：已管理级 Managed
 - 项目小组体现出项目管理的特征，有项目计划和跟踪、需求管理、配置管理等
- 等级 3：已定义级 Defined
 - 公司层面有标准流程和相应的规范，每个项目小组可以基于此定义自己的过程，使得优秀的做法可以在公司内分享。
- 等级 4：定量管理级 Quantitatively Managed

- 构建**预测模型**，已统计过程控制的手段来管理过程
- 等级 5：优化级 Optimizing
 - 继续应用**统计方法**识别过程偏差，找到问题根源并消除，避免未来继续发生类似问题
- 等级 2 和等级 3 关注的是当前状态
- 等级 4 和等级 5 是根据结果（未来）来进行管理
- 高等级原因：定量管理和改进
- 本质差别：通过数据和模型来管理和改进过程

8. 敏捷宣言（背）

敏捷价值观：

- 个体和互动 胜过 流程和工具
- 可以工作的软件 胜过 详尽的文档
- 客户合作 胜过 合同谈判
- 响应变化 胜过 遵循计划

尽管右项有其价值，我们更重视左项的价值。

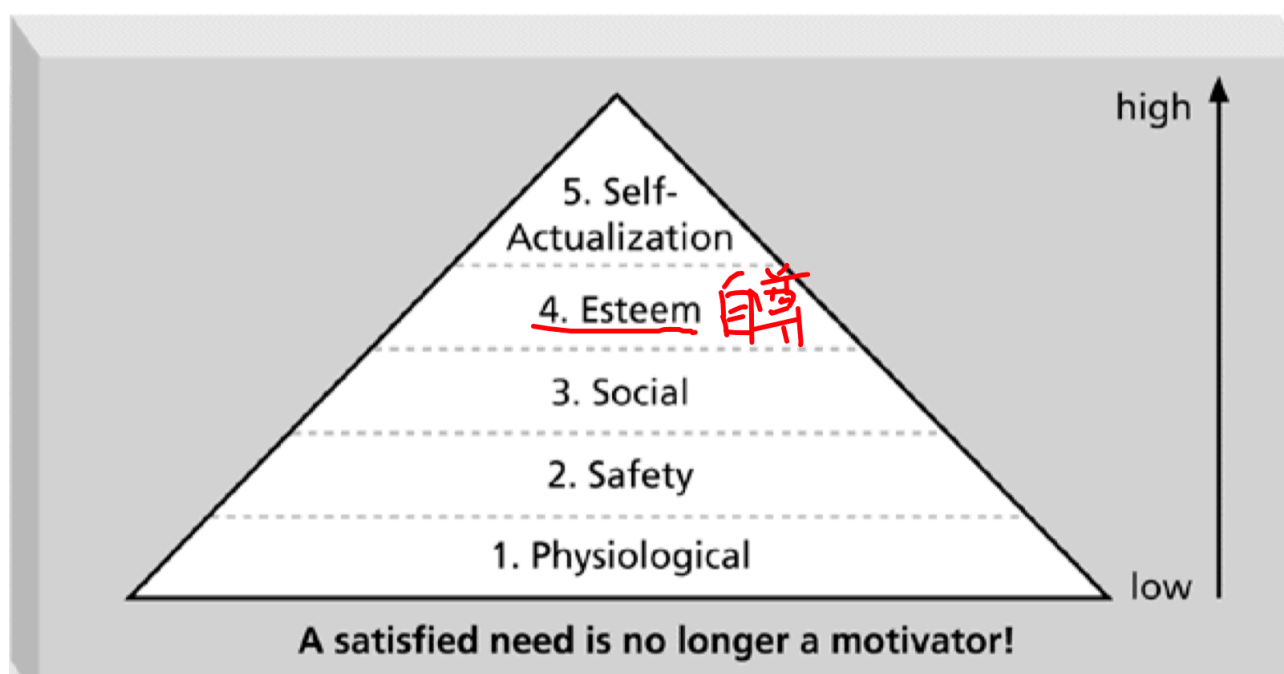
以下都是敏捷方法：

- XP (eXtreme Programming) ， 极限编程
- SCRUM：管理框架和管理实践
- Kanban 看板

9. 领导者的激励手段

- 威逼、利诱、鼓励承诺

10. 马斯洛的需求层次理论



- 自我实现是最高层次
- 激励来自为没有满足的需求而努力奋斗

- 低层次的需求必须在高层次需求满足之前得到满足
- 满足高层次的需求的途径比满足低层次的途径更加广泛

11. 海兹伯格的激励理论

- 激励因素（内在因素）
 - 成就感，责任感，晋升，被赏识、认可
- 保健因素（外在因素）
 - 工作环境，薪金，工作关系，安全等

12. 麦克勒格的 X-理论

- 不喜欢他们的工作并努力逃避工作
- 缺乏进取心，没有解决问题与创造的能力
- 更喜欢经常的指导，避免承担责任，缺乏主动性
- 自我中心，对组织需求反应淡漠，反对变革
- 用马斯洛的底层需求（生理和安全）进行激励

13. 麦克勒格的 Y-理论

- 如果给予适当的激励和支持性的工作氛围，会达到很高的绩效预期
- 具有创造力，想象力，雄心和信心来实现组织目标
- 能够自我约束，自我导向与控制，渴望承担责任
- 用马斯洛的高层需求（自尊和自我实现）进行激励

14. 软件开发和自主团队（背）

软件开发特点

- 软件开发是一项既复杂又富有创造性的知识工作
- 软件开发是一种智力劳动
 - 处理和讨论极其抽象的概念
 - 把不同的部分（不可见）整合成一个可以工作的系统
 - 全身心地参与
 - 努力做出卓越的工作
- 软件开发是一种智力活动，开发者是智力劳动者，而对于智力劳动者而言，管理的第一准则，就是智力劳动者不能被管理，只能实现自我管理。（自主团队的必要性）

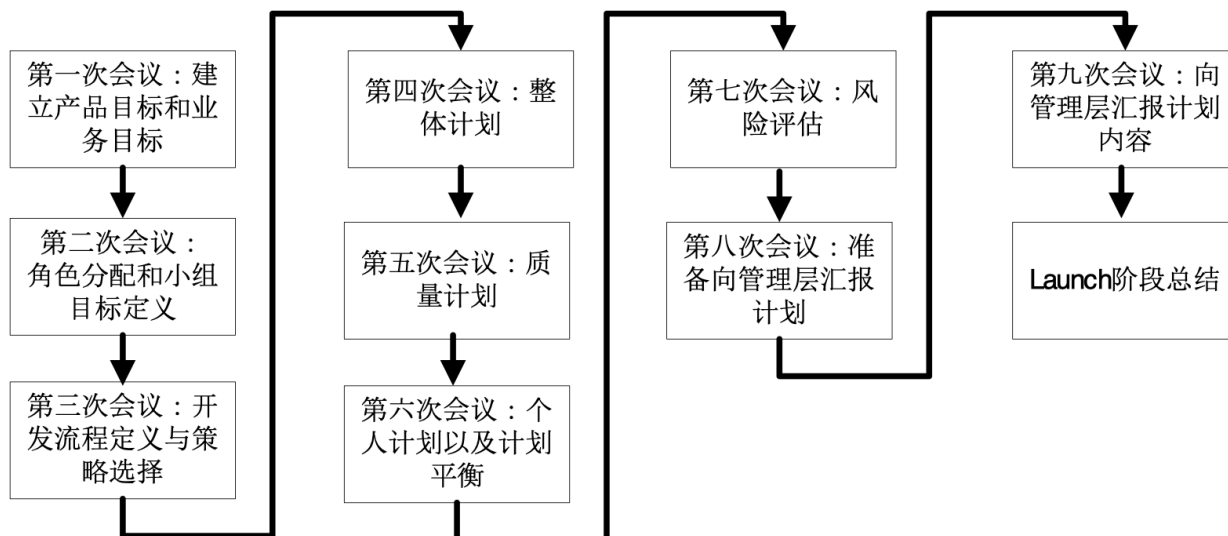
自主团队的特点

- 自行定义**项目的目标**
- 自行决定**团队组成形式以及成员的角色**
- 自行决定项目的**开发策略**
- 自行定义项目的**开发过程**
- 自行制定项目的**开发计划**

- 自行度量、管理和控制项目工作

15. TSP Launch (背)

- TSP: Team Software Process, 团队软件过程
- PSP: Personal Software Process, 个人软件过程



TSP 角色和职责

- **项目组长** (项目经理)：建设和维持高效率的团队
- **计划经理**：开发完整的、准确的团队计划和个人计划
- **开发经理**：开发优秀的软件产品
- **质量经理**：保证项目团队严格按照质量计划开展工作，开发出高质量的软件产品
- **过程经理**：维护所有团队成员准确的记录、报告和跟踪过程数据
- **支持经理**：保证项目小组在整个开发过程中都有合适的工具和环境
- **开发人员**：开发

16. SCRUM 小组角色

典型 SCRUM 团队由一名产品负责人、开发团队和一名 SCRUM Master 组成。

17. PROBE 方法 (背)

全称叫 PROxy Based Estimation

作用

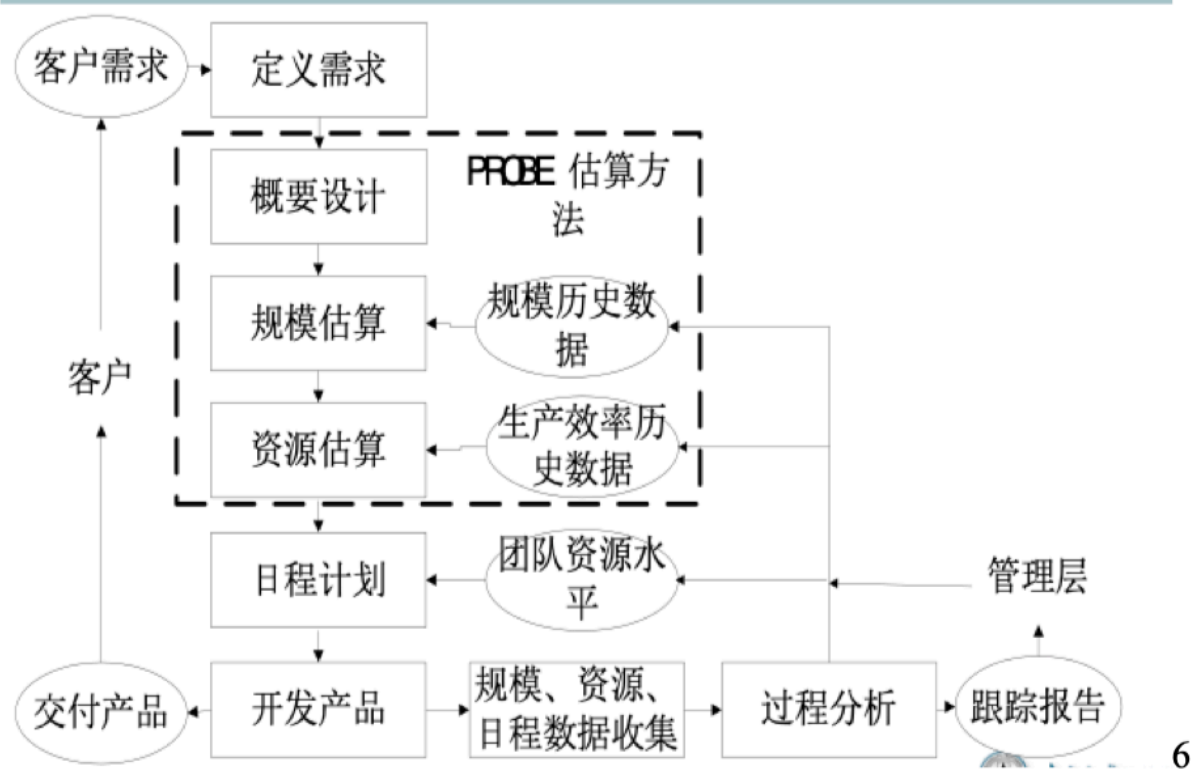
精确度量和早期规划之间的桥梁

原理

1. 设立合理的代理作为**精确度量**和早期规划需要的度量之间的桥梁
2. 相对大小，而非绝对大小

(PROBE 是可以说精确度量的，一般来讲使用功能点估算)

通用计划框架中的 PROBE 方法



虚线框中即为 PROBE，用来完成规模和资源的估算。2

应该要做哪些估算？

应该要做 规模估算 和 资源估算。

PROBE 方法充当了什么角色？

精确度量与早期规划之间的桥梁

虚线框中即为 PROBE，用来完成规模和资源的估算。

简要解释应用 PROBE 方法估算的优缺点

PROBE 方法通过定义的估算过程和数据收集以及使用的框架，使得估算结果可以尽可能一致，从而使得一些统计方法可以用来调整估算结果，增强用户对估算结果的信心。

解释为什么使用 PROBE (A) 方法估算时间的时候，为什么不使用历史数据中的生产效率数据？

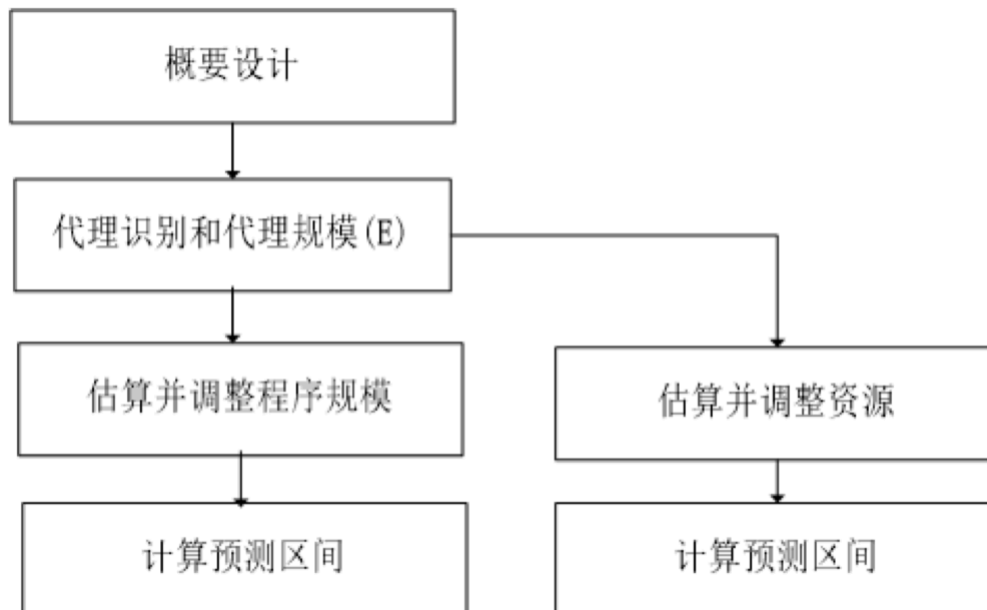
在估算资源需求（例如，人时）的时候，生产效率一般在分母上，考虑到个人软件工程师的生产效率波动，容易导致估算偏差范围变大。

描述一下 PROBE 方法的基本原理和过程

原理

- 设立合理的代理作为精确度量和早期规划之间的桥梁
- 相对大小而非绝对大小

过程



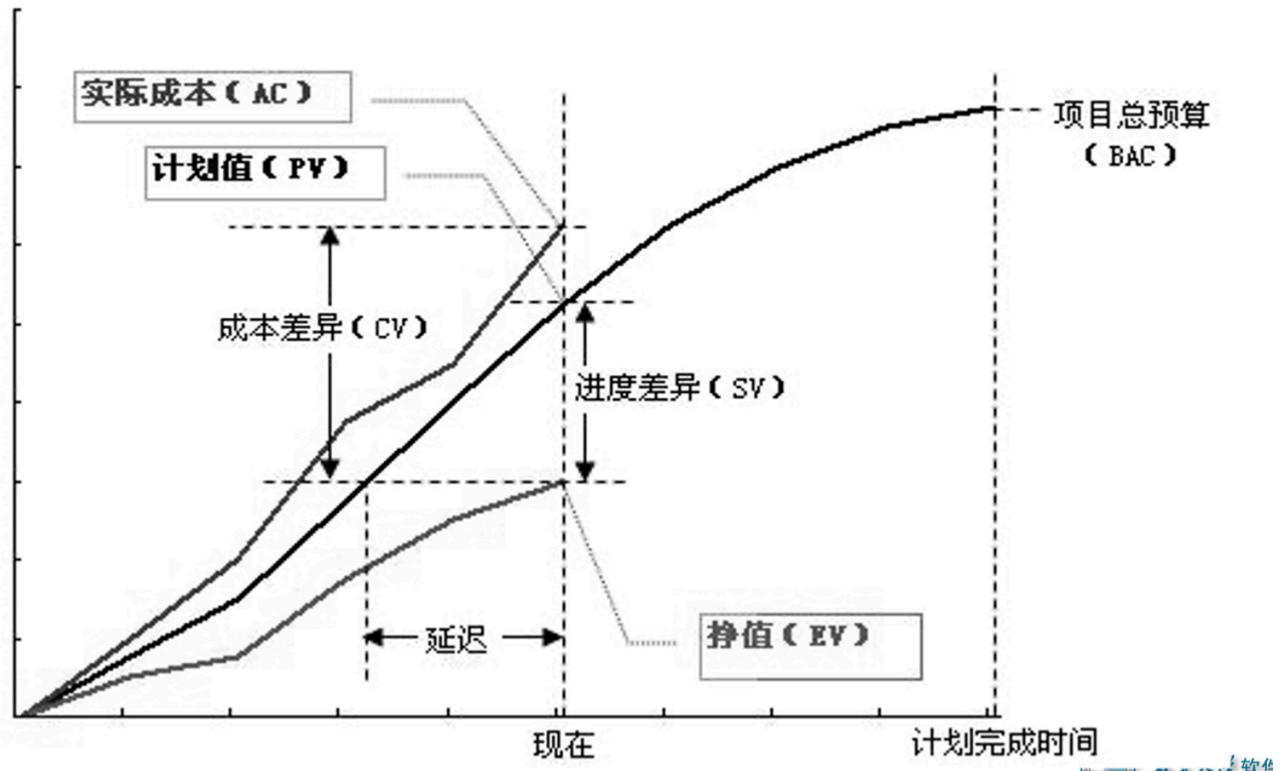
18. 估算的要点（背）

1. 尽可能划分详细一些
2. 建立对结果的信心
3. 依赖数据
4. 估算要的是过程，而非结果；估算的过程是相关干系人达成一致共识的过程

19. 挣值管理方法（背）

- EVM, Earned Value Management 是用来客观度量项目进度的一种项目管理方法。
- EVM 采用与进度计划、成本预算和实际成本相联系的三个独立的变量，进行项目绩效测量。
 - 简单实现
 - 中级实现
 - 高级实现

挣值分析图示



- 可能是：考虑成本差异高一级，再考虑计划完成时间又高一级，存疑
- chatgpt:
- 在挣值管理中，初级、中级和高级实现是指在实施挣值管理过程中的不同层次和复杂度。以下是它们之间的区别：

1. 初级实现：

初级实现是指基本的挣值管理实践，通常适用于较小规模、较简单的项目。在初级实现中，主要关注计划价值 (PV)、挣值 (EV) 和实际成本 (AC) 这三个基本指标的计算和跟踪。项目经理和团队会定期收集和记录实际工作完成情况和实际成本，并计算出进度偏差 (SV) 和成本偏差 (CV)。初级实现通常使用基本的电子表格工具来进行数据记录和计算。

2. 中级实现：

中级实现在初级实现的基础上进一步扩展，可以应用于具有较大规模、较复杂的项目。除了 PV、EV、AC、SV 和 CV 之外，中级实现还引入了一些更高级的指标和分析方法。例如，引入了**进度绩效指数 (SPI)** 和**成本绩效指数 (CPI)** 来评估项目的绩效效率。此外，中级实现还可以考虑使用挣值图表和趋势分析来可视化和跟踪项目的进度和成本绩效。

3. 高级实现：

高级实现是最复杂和全面的挣值管理实践，适用于大规模、复杂的项目和组织。除了 PV、EV、AC、SV、CV、SPI 和 CPI 之外，高级实现引入了更多的高级指标和技术。例如，引入了**挣值指数 (EI)** 和**估计完成指数 (ETC)** 来预测项目的完成情况和成本。此外，高级实现还可以结合其他项目管理方法和工具，如风险管理、资源管理和质量管理，以实现更全面的项目控制和决策支持。

总体而言，初级实现主要关注基本的挣值管理指标的计算与跟踪，中级实现在此基础上引入了更多的指标和分析方法，而高级实现则提供了更全面和高级的技术和工具，以支持复杂项目的控制和决策。实际选择哪个层次的实现取决于项目的规模、复杂度和需求，以及组织的成熟度和资源可用性。

简单实现

这种方式仅仅关注进度信息。在实现时，首先需要建立WBS，定义工作范围；其次为WBS中每一项工作定义一个价值（PV）；最后按照一定的规则将某一数值赋给已经完成的工作或者正在进行的工作。常用规则分别为0-100规则和50-50规则，前者只有当某项任务完成时，该任务的PV值将转化成EV值；后者只需要开始某项任务，即可以赋原PV值的50%作为EV值，完成时，再加上另外的50%。而实际完成的工作所需成本AC不对EV值产生任何影响。

中级实现

在简单实现的基础上，加入日程偏差的计算。典型计算方式有：

日程偏差 $SV = EV - PV$ ；

日程偏差指数 $SPI = EV / PV$ ；

高级实现

在中级实现的基础上，还需要考察项目的实际成本。

TSPi Week Summary - Form WEEK

Project: Cassiopeia

PM:

Name:

--Team--

WeeklyData	Plan	Actual	Plan / Actual	Plan - Actual
Hours for this week	144.09	121.83	1.18	22.25
Hours to date	246.48	286.03	0.86	-39.55
Earned value for this week	42.75	36.84	1.16	5.91
Earned value to date	73.13	81.29	0.9	-8.16
To-date hours for tasks completed	246.48	286.03	0.86	
To-date average hours per week	82.16	95.34	0.86	
EV per completed task hour to date	0.3	0.28		

- 看 to date 可以看出目前的情况。
- 先看 earned value to date, $Plan < Actual$ ，说明至今进度没有落后
- 再看 Hours to date, 所用时间 $Actual > Plan$ ，earned value per completed task hour to date 里面，每小时应该创造 0.3 的价值，但是实际上只创造了 0.28 的价值，项目开发有隐患，再过几周可能会逐渐落后。

挣值管理有三种实现方式。分别是简单、中级以及高级。请分别概述上述三种方式的基本要点。

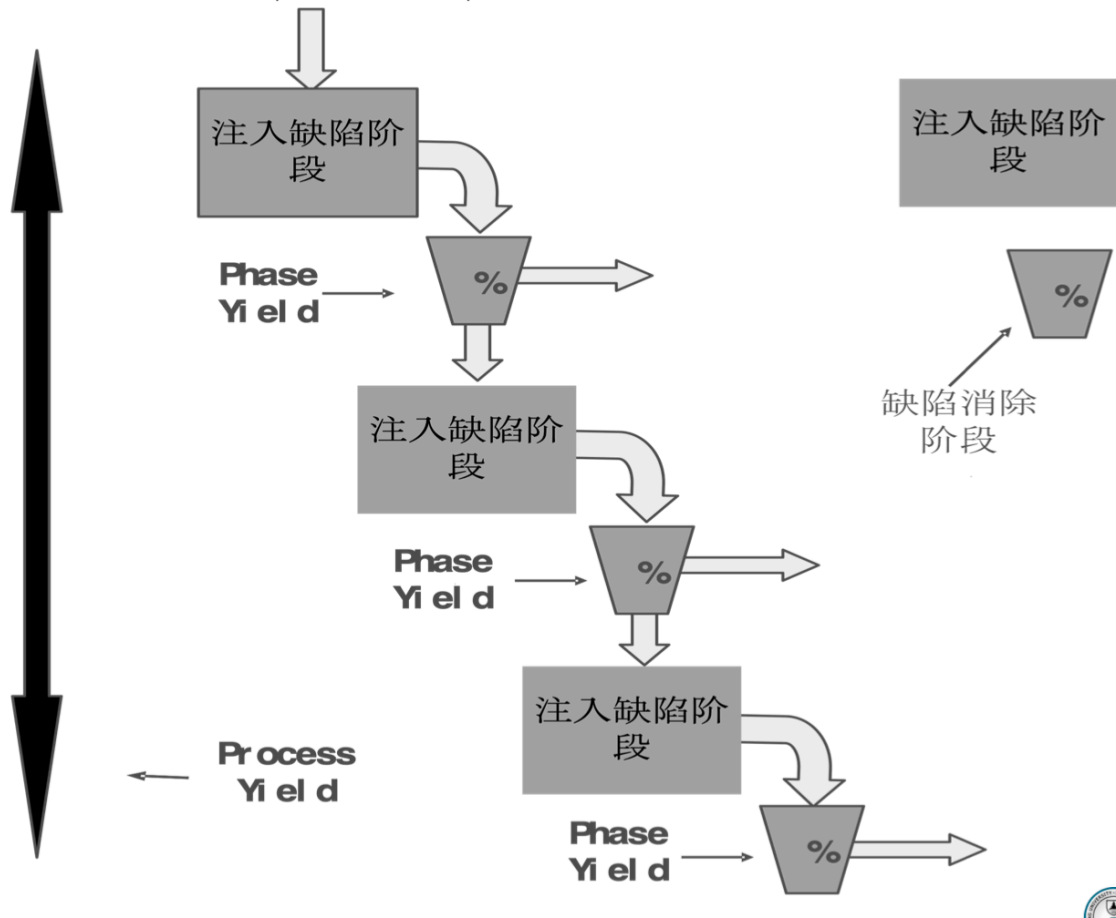
- 简单实现：
 - 首先需要建立 WBS，定义工作范围
 - 其次为 WBS 中每一项工作定义一个价值 PV
 - 最后按照一定的 100-0 或 50-50 规则将某一数值赋给**已经完成的工作**或者**正在进行的工作**，就是工作的价值 EV
- 中级实现：在简单实现的基础上，加入**日程偏差**的计算
- 高级实现：在中级实现的基础上，加入**成本差异**的计算
- 简单实现有 PV 和 EV 两条线，中级实现增加以 BAC 项目总成本为代表的预估（有多项预估，包含日程偏差指数 SPI），高级实现增加成本预估（不仅仅 AC）

20. 质量指标：Yield、A/FR、PQI、Review Rate、DRL

- wfs：应该先编译后评审，评审由于速度不能太快，评审放编译前后都一样，但是先编译可以除掉很多错，就先编译

Yield

- $\text{Phase Yield} = 100 * (\text{某阶段发现的缺陷个数}) / (\text{某阶段注入的缺陷个数} + \text{进入该阶段前遗留的缺陷个数})$
- $\text{Process Yield} = 100 * (\text{第一次编译前发现的缺陷个数}) / (\text{第一次编译前注入的缺陷个数})$
- process 是整体，若干个 phase 组成一个 process



A/FR

- $A/FR = \text{PSP 质检成本} / \text{PSP 失效成本}$
 - PSP 中定义的失效成本为**编译时间**和**单元测试时间**之和
 - PSP 中定义的质检成本为**设计评审时间**与**代码评审时间**之和
- 理论上 A/FR 的值越大，往往意味着越高的质量。
- 过高的 A/FR 往往意味着做了过多的评审，反而会导致开发效率的下降。
- 在 PSP 中 A/FR 的期望值就是 2.0

PQI

五个数据乘积

- 设计质量：设计的时间应该大于编码的时间
- 设计评审质量：设计评审的时间应该大于设计时间的 50%
- 代码评审质量：代码评审时间应该大于编码时间的 50%
- 代码质量：代码的编译缺陷密度应当小于 10 个 / LOC 千行
- 程序质量：代码单元测试缺陷密度应当小于 5 个 / 千行

PQI 的期望值是 0.4，如果 $PQI > 0.4$ ，则可认为软件产品的质量较高

Review Rate

- 高质量的评审需要软件工程师投入足够多的时间进行评审
- 在 PSP 的实践中，代码评审速度小于 200 LOC / 小时，文档评审速度小于 4 Page / 小时

DRL

- DRL 指的是缺陷消除效率比 Defect Removal Efficiency，度量的是不同缺陷消除手段消除缺陷的效率
- 计算方式是某个测试阶段（一般是单元测试）每小时发现的缺陷数为基础，其他阶段每小时发现缺陷数与该测试阶段每小时发现的缺陷的比值。

阶段	缺陷个数/小时	DRL (UT)
设计评审	3.6	1.03
编码评审	8	2.29
单元测试	3.5	1

21. 质量路径 Quality Journey

为了追求极高的质量，你有哪些手段？

- 各种测试
- 进入测试之前的产物质量提升
- 评审过程度量 and 稳定
- 质量意识和主人翁态度
- 个体 review 的度量和稳定
- 诉诸设计
- 缺陷预防
- 用户质量观 —— 其他质量属性

22. PSP 设计模板

- 操作规格模板 (Operational Specification Template, OST)
 - 描述的是系统与外界的交互，用户与待设计系统的正常情况和异常情况下的交互
 - 可以用来定义测试场景和测试用例，也可以作为和系统用户讨论需求的基础，特别是操作相关的需求描述
- 功能规格模板 (Functional Specification Template, FST)
 - 描述的是系统对外的接口，是一种静态信息的描述
 - 消除二义性非常重要，尽可能使用形式化符号来描述方法等行为
- 状态规格模板 (State Specification Template, SST)

- 可以精确定义程序所有的状态和转移以及伴随着每次状态转换的动作
- 逻辑规格模板（Logical Specification Template, LST）
 - 精确描述系统的内部静态逻辑
 - 为了消除描述的二义性，一般建议用伪代码配合形式化符号来描述计算结果

	动态信息	静态信息
外部信息	交互信息（服务、消息等）	功能（继承、类结构等）
内部信息	行为信息（状态机）	结构信息（属性、业务逻辑等）

	动态信息	静态信息
外部信息	OST/FST	FST
内部信息	SST	LST

23. 设计验证方法

- 状态机验证
 - 正确的状态机应该是：完整、正交的
- 符号化执行验证
 - 基本思想是将描述设计的逻辑规格（一般用伪代码程序表示）用代数符号来表示，然后**手工**分析伪码程序的行为
 - 优点是：
 - 实施简单，可以给出一般化的验证结果，很多时候往往是唯一提供全面验证的方式
 - 通常用在验证一些复杂算法中，特别是对遗留系统的改造中，往往应用这种方法来识别和理解原有的设计
 - 缺点是：
 - 不适用于有复杂逻辑的场合
 - 纯手工的验证方法也容易引入一些人为的错误
- 执行表验证
 - 采用构建执行表的方式跟踪伪码程序的执行状况，分析程序行为

- 跟踪表验证
 - 是对执行表验证方法的一种扩充，区别在于
 - 会识别将伪码程序符号化的机会，并加以符号化
 - 定义并且优化用例组合
- 正确性检验
 - 定理证明？把伪码程序方程数学推理，采用形式化方法加以推理和验证
 - 对于复杂伪码程序的结构，应用正确性检验的标准问题逐项加以验证
 - 对于不能明确判断的复杂程序结构，使用跟踪表等辅助验证

24. 集成策略选择

- 大爆炸集成策略
- 逐一添加集成策略（持续集成本质上就是逐一添加策略）
- 集簇集成策略
- 扁平化集成策略

25. 验证与确认 V & V

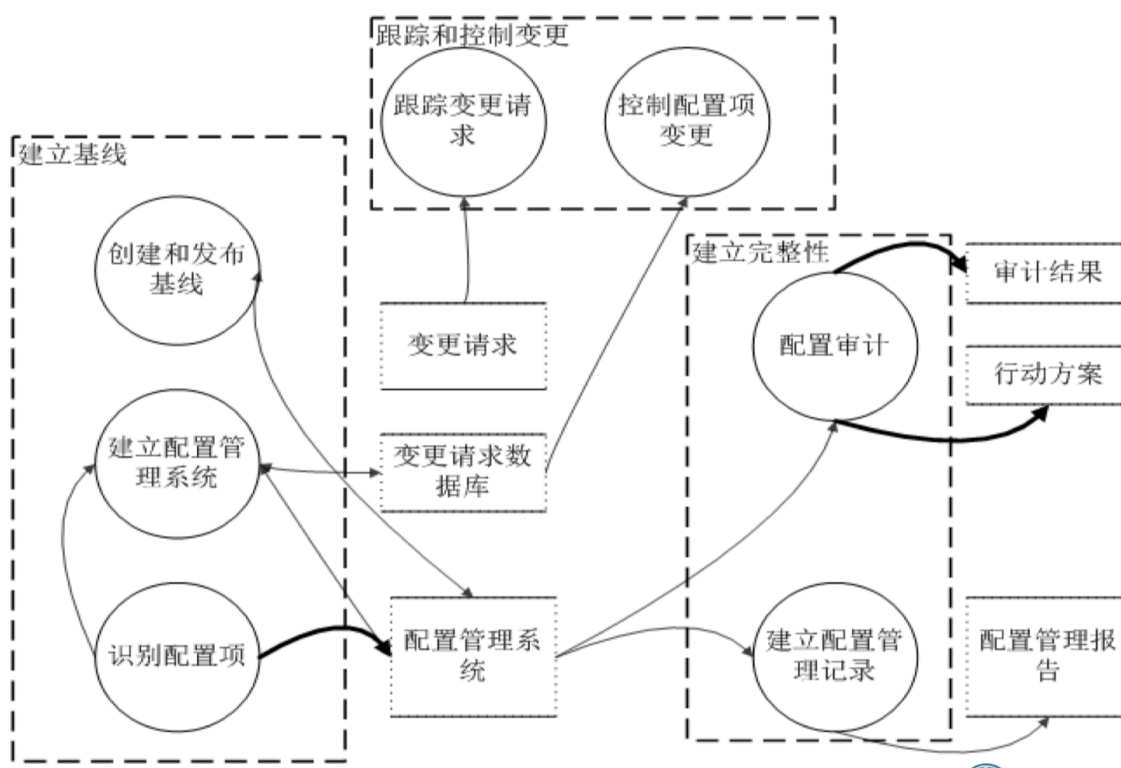
- 验证 Verification、确认 Validation 都是为了提升最终产品的质量而采取的措施
- 验证和确认的目的不同
 - 验证是目的是确保选定的工作产品与事先指定给该工作产品的**需求**一致
 - 确认的目标则是确保开发完成的产品或者产品组件在即将要使用该产品或者产品组件的**环境**中工作正确

26. 配置管理

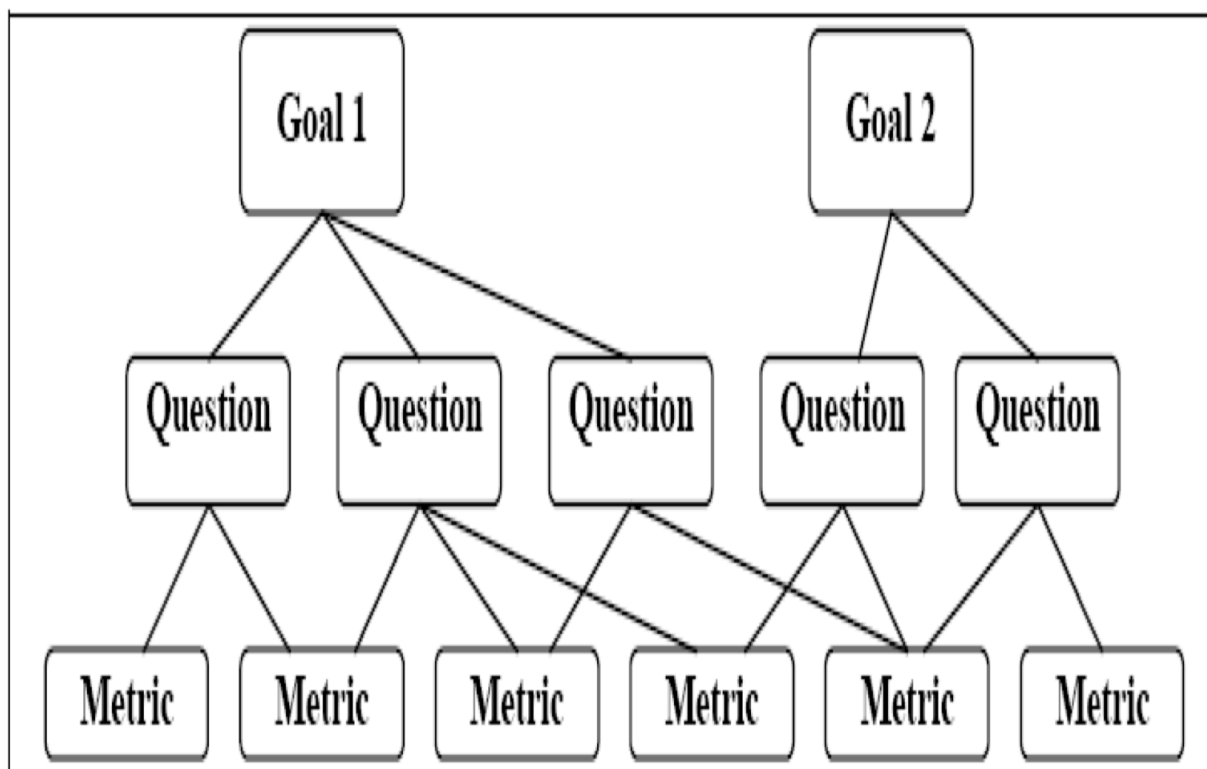
- 目的是建立与维护工作产品的完整性

- 配置管理活动

配置管理活动



27. 度量与分析活动 GQM 方法



28. 根因分析

- 避免类似错误反复发生
- 一个正式根因分析往往包含下列的活动

- 识别和选定问题
- 根因分析
- 建立改进的行动方案
- 实施改进，评估效果
- 典型角度
 - 技术角度、人员角度、培训角度、过程角度

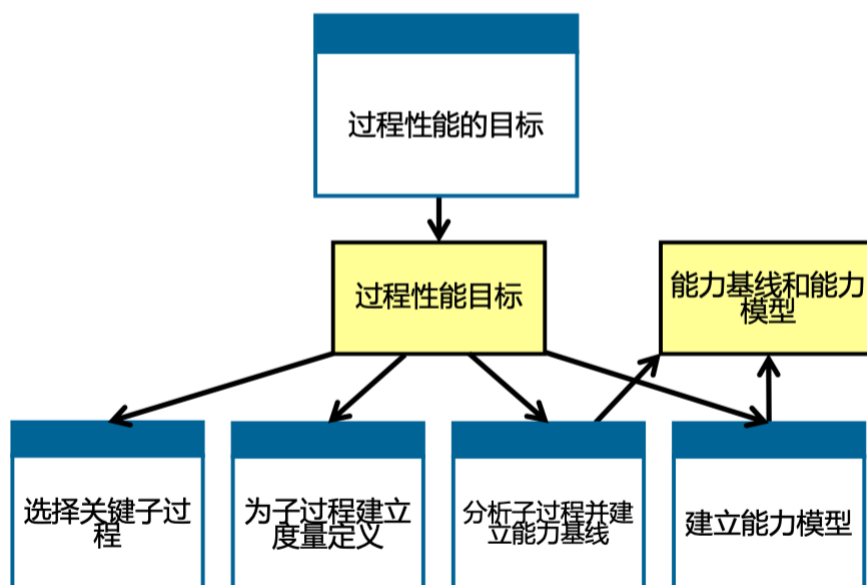
29. Yield 预测模型

设计一个 Yield 预测模型

1. 描述如何建模
2. 需要哪些数据，有什么要求
3. 用什么方法构建模型并预测

1. 答定量模型构建的关键实践

定量模型构建的关键实践



1. 识别**过程性能目标**
 2. 选择**关键子过程**
 3. 为子过程建立**度量定义**
 4. 分析子过程并建立**能力基线**
 5. 建立**能力模型**
2. 需要哪些数据：
 1. **注入阶段**注入多少缺陷
 2. 缺陷注入的**密度**（需求每一页注入多少缺陷）
 3. 缺陷注入的**速度**（每小时注入多少缺陷）
 4. **消除阶段**的缺陷注入密度和速度

5. 历史数据中的软件规模、文档规模、开发人员规模（其他被控制的自变量）
3. 可能的改进方案：假设注入水平和消除水平都符合**正态分布**，因此，可以用**蒙特卡罗方法**模拟结果

11. 下列关于各种设计验证手段的描述中正确的是：

- A. 执行表是唯一一种提供全面设计验证的手段；（符号化验证才是）
- B. 跟踪表是唯一一种提供全面设计验证的手段；（符号化验证才是）
- C. 受限于手工方式，都易于出错；**
- D. 符号化执行验证不适合复杂的计算过程；（适合复杂的计算过程，不适合复杂逻辑）

12. 下述关于质量的描述中，哪些说法不正确的？

- A. 质量是一种多重属性的组合
- B. 最终用户一般不能感知内部质量
- C. 安全和保密一般不是质量要素**
- D. 质量与主观感受有关

13. 下述关于质量控制指标，哪些说法正确？

- A. A/FR应该是越高越好（2.0就好，太高了代价大）
- B. Yield是一种精确度量模块质量的手段（估计度量）
- C. 评审活动应该早于编译或者测试活动而开展**（评审放前还是放后的时间开销是一样的，但是先评审可以改错，减少编译和测试的时间）
- D. PQI只能事后统计，不能用于指导质量计划（PQI可以预测和度量，可以指导质量计划）

14. 下述设计验证手段的描述，哪些是正确的？

- A. 符号化执行容易引入人为错误**
- B. 状态机验证是唯一一种提供一般意义的上的正确性检验的验证手段（至少还有符号化验证）
- C. 执行表的对设计缺陷的验证能力强于跟踪表（弱于跟踪表，跟踪表是执行表的扩展）
- D. 正确性检验是唯一可靠的设计验证手段（真值表也是）